Comparing BMAD and GitHub Spec Kit for Spec-Driven Development: A Requirements Engineering Case Study on a Java Swing Calculator

Varun Ganesh Sondur
Tampere University

Mohammadamin Lotfiourimi
Tampere University

Pham Thanh
Tampere University

Georgios Malezoglou
Tampere University

Abstract—Spec-driven development (SDD) tools let coding agents work from written specifications instead of chat instructions. We ask how two such tools, BMAD and GitHub Spec Kit, support the step from requirements to implementation and the handling of a requirement change. Using a comparative case study, two team members built the same Java Swing calculator, one with each tool, from one frozen set of requirements (8 functional and 5 non-functional requirements, 19 acceptance criteria). The same change request, which adds square, square root and reciprocal operations (11 requirements, 22 criteria), was then given to both. Independent evaluators later ran common execution sheets. Both workflows reported all 22 change criteria and all 19 baseline re-checks as passed. The independent baseline evaluation gave BMAD 19 passes and Spec Kit 18 passes and one blocked check (an offline-launch prerequisite), which is a difference in evidence, not in demonstrated correctness. Spec Kit's independent change evaluation left more inspectable raw evidence. Effort, change-impact recall, native traceability and rubric ratings were not measured, so we make no claim about an overall winner. The study has one case, one run per workflow and confounded operator, agent and environment.

Index Terms—spec-driven development, requirements engineering, traceability, change impact analysis, BMAD, GitHub Spec Kit

I. Introduction

Coding agents now produce working software from short instructions, but the intent behind that output is easily lost when the instructions live only in a chat [1]. Spec-driven development (SDD) responds by making the specification an operational input: a person writes or approves a specification, a plan and a task list, and the agent generates code against them [2], [3]. Tools such as GitHub Spec Kit [4] and BMAD [5] package this idea into installable workflows.

From a requirements engineering (RE) point of view, the interesting question is not which tool generates better code. It is how each workflow carries requirements through specification, design, tasks, code and tests, whether those links can be followed, what happens when a requirement changes, and how much of the original requirement set the result satisfies. The sources we read describe SDD practices and test claims about them at repository scale [1], [3], [6], but they do not compare two named SDD tools on one controlled software case with a common change request.

This report presents such a comparison as a small, bounded case study. Two members of our team each operated one tool to build the same desktop calculator from the same frozen requirements, then applied the same change request. Two other members evaluated the results independently with shared checklists. The study is descriptive: the unit of comparison is the whole tool, agent and operator workflow, and we do not claim to isolate the effect of either tool.

We make three contributions. First, we describe both tools as they were installed and used, with their native artefacts and change paths. Second, we report acceptance outcomes, change-impact observations and process observations for the baseline and the change, and we state for every number which evidence layer it comes from. Third, we list which planned measures could not be calculated and why. The report first gives the background and the method, which includes the research questions. It then describes the tools, presents the results, discusses the questions and the threats to validity, and ends with a conclusion and the AI-use declaration.

II. Background

A. Spec-Driven Development

SDD treats the specification as the source of truth and the code as a generated or verified secondary product. Piskala distinguishes three levels of specification rigour: spec-first, where a specification is written before the code; spec-anchored, where it is kept alongside the code; and spec-as-source, where the specification is the main artefact [2]. We use this vocabulary only to describe the workflows. We do not rank the tools by it; the abstract does not present it as a measure for comparing tools.

Claudino et al. scored specification, plan and task artefacts of 114 open-source SDD repositories against ISO/IEC/IEEE 29148 and derived a framework of 40 practices in 10 quality dimensions [3]. Agarwal et al. combined a thematic analysis, 21 practitioner interviews and a corpus of 470,795 specification files from 266 repositories; they report that implementation detail in specifications may help agents but makes specifications more likely to go stale, and that specification and code are usually checked against each other only when a change is finished [1]. Hill

analysed 88,052 pull requests in 119 repositories and found no meaningful difference in defect introduction between specified and unspecified pull requests after matching; the specification artefacts in that sample were mostly references to issues and tickets, and the author notes that the effect of adopting SDD tools remains untested [6]. We read only the abstract pages of these three works, and two of them are preprints whose peer-review status we have not established. They justify caution: a specification is not automatically a quality improvement.

Vibe coding, meaning prompting an agent without a written specification, is named in our project brief as a contrasting approach. It is not an experimental condition here.

B. Requirements Engineering Concepts Used

We use four RE concepts as operational definitions. Traceability is the ability to follow a requirement forward to design, tasks, code and tests and back. We separate links created by the workflow itself (native links) from links reconstructed afterwards by an evaluator, because the second kind says little about what the workflow supports. Change-impact analysis is the statement, made before coding, of which requirements, design elements, code and tests a change will touch. Acceptance criteria are checks that turn a requirement into something an evaluator can observe. Non-functional requirements (NFRs) are quality constraints such as launch behaviour or the absence of network access; in a small GUI application they are the hardest to turn into observable criteria.

C. Related Work

We did not conduct a systematic literature search; the sources above were found through general web searches and the reference lists of other papers. Within them, the work closest to ours analyses repositories at scale [1], [3], [6]. Our study differs in design: it is small, runs two named tools on one shared case, and applies one requirement change to both. Its methodological basis is guidance for software engineering case studies [7] and for measurement and data collection in empirical studies [8]. Measures are organised with the Goal Question Metric approach [9]. We read the bibliographic records of these three sources, not their full texts.

III. Method

A. Research Questions

Our main question is: How do SDD tools support the transition from requirements to implementation, and how well does the result match the original requirements? It is split into four questions.

- RQ1. How does each tool create and structure specifications, including formats, levels of detail, templates and the role of the human?
- RQ2. How are requirements turned into design, tasks, code and tests, and is the link between these artefacts traceable?
- RQ3. How does each tool handle changes to the specification, and does the specification remain aligned with the code?
- RQ4. How closely does the generated implementation satisfy the original requirements?

The team's later analysis used a different set of five questions that ask which workflow performed better, handled the change better and produced better documentation. We map its findings onto RQ1 to RQ4 and do not adopt those questions as headings.

B. Case and Change Request

The case is a desktop calculator in Java Swing with two operand fields, four operation buttons, a Calculate and a Clear button, and inline error messages. Arithmetic is exact in decimals; division rounds half-up to ten places; results are shown as plain decimal text. The error messages are fixed, and the error priority is empty input, then no operation, then an invalid number, then an arithmetic error such as division by zero. The baseline requirements were frozen on 2026-10-23.

The change request, CR-01, adds three unary operations: square, square root and reciprocal. The First number field is used and the Second number is ignored. The square is exact; square root and reciprocal are rounded half-up to ten places; a negative square root and a reciprocal of zero are errors; and the criteria require all baseline checks to be re-run. Table I gives the size of the case.

TABLE I
Size of the shared case per phase.

Phase	Requirement IDs	Parent acceptance criteria	Re-checked baseline parents
Baseline	8 functional + 5 non-functional	19 (BASE-AC-01..19)	n/a
CR-01	11 functional (CR-FR-01..11)	22 (CR-AC-01..22)	19

The case has known limits. Four baseline NFRs have no directly linked acceptance criterion and three criteria have no requirement ID. This is a limitation of the shared input and applies to both tracks.

C. Experimental Procedure

Fig. 1 shows the design. Each track started from the same frozen requirements and acceptance criteria and kept its tool's native workflow. The fairness rules were: the same inputs for both, no sharing of generated artefacts between tracks, native workflows left intact, human interventions recorded and every deviation documented. Shared ambiguities and decisions were recorded in a

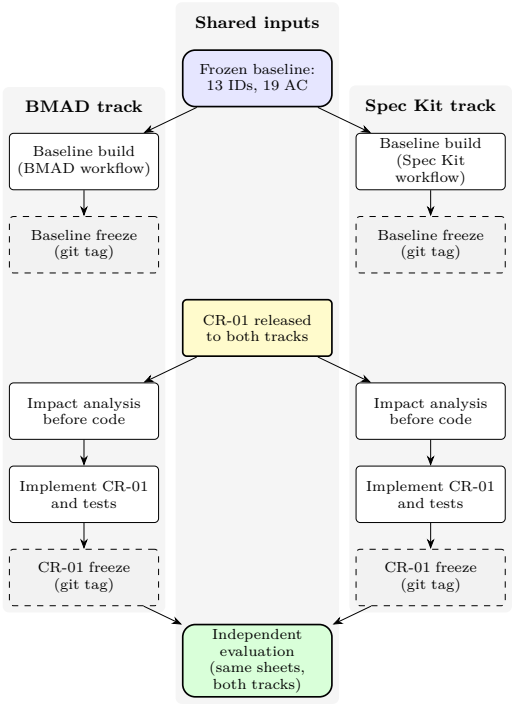


Fig. 1. Study design: one frozen baseline and one change request (CR-01) given to both tracks, followed by independent evaluation.

common experiment log, which both tracks had to follow. Each track froze its baseline with a git tag before CR-01 was released. For CR-01, each track produced a pre-coding impact analysis, implemented the change and froze the result with a second tag. The shared baseline snapshot was baseline-frozen; the BMAD change snapshot is the commit tagged bmad-cr01-frozen.

Two members who had built neither calculator evaluated the results with identical execution sheets for the baseline criteria and the change criteria. One evaluated the BMAD track and one evaluated the Spec Kit track; the Spec Kit baseline evaluation was run with assistance from an AI coding assistant. For Spec Kit, the evaluation used the baseline snapshot designated by the study owner, because the track's own operator results had been recorded at an earlier commit and the track had not declared a freeze; the log records that differences between the two commits may exist. All sources for this report, including both tracks' evidence and the experiment log, are in the team repository.

D. Evaluation Approach and Measures

We report acceptance outcomes as Pass, Fail, Blocked or Not run. A criterion counts as Pass only if the behaviour was observed. A Blocked criterion is not a Fail, a Not run criterion is not a zero, and a later manual completion does not replace the earlier Blocked record. Three layers of evidence are kept apart: operator reports, independent evaluator observations, and native automated tests. Counts of native tests are context only, because the two tracks use different test designs, layers and environments. Following Runeson and Höst, we describe validity threats by category rather than hiding them in an overall score [7], and following Kitchenham et al. we state counting rules explicitly [8].

The measures come from a catalogue of sixteen measures (M01 to M16) that the team derived from five evaluation goals with the Goal Question Metric approach [9]. Those that matter here are acceptance outcomes (M01), native and reconstructed traceability (M03, M13), defects (M04), regressions after the change (M06), change-impact recall (M07), interventions and rework (M08, M12), effort and duration (M09, M10), rubric ratings (M11) and native test inventory (M16). Table V in the Results states which could be calculated.

IV. Tool Descriptions

Both tools were used through the same coding agent, Claude Code, with the model claude-sonnet-5-5. Table II records the environments. The two environments differ in operating system, operator and, at the start of the Spec Kit track, in the model.

TABLE II
Environment and versions per track.

Item	BMAD track	Spec Kit track
Operating system	macOS 15.7.1 (arm64)	Windows 11 Home
JDK (pinned via JAVA_HOME)	Amazon Corretto 22.0.2	Amazon Corretto 22.0.2.9.1
Build tool	Maven 3.9.16	Maven 3.9.16
GUI toolkit	Swing	Swing
Test framework	JUnit Jupiter 6.1.3	JUnit Jupiter 6.1.3
Tool and version	BMAD 6.12.4 (core; bmm)	specify-cli 1.2.3 (claude integration)
Coding agent	Claude Code	Claude Code (VS Code extension)
Model	claude-sonnet-5-5	claude-opus-5-5 at start; then claude-sonnet-5-5
Operator	Varun	George

A. BMAD

BMAD, expanded by its maintainers as the Breakthrough Method for Agile AI-Driven Development, is an open-source framework that covers deciding what to build, how the parts fit and how the work changes as one learns [5]. Its documentation describes a delivery loop of clarifying a vague idea, planning a large idea, building and verifying a small change, and learning and adjusting. It offers specialised perspectives for product, architecture, UX, development and testing. The installed native flow in our track ran from an optional product brief to a PRD, an optional UX step, an architecture, epics and stories, sprint planning, build, and optional code review and QA.

Fig. 2 shows the path actually followed. The operator used a fast path for the PRD and architecture, skipped the UX step, and had the agent judge the readiness gate.

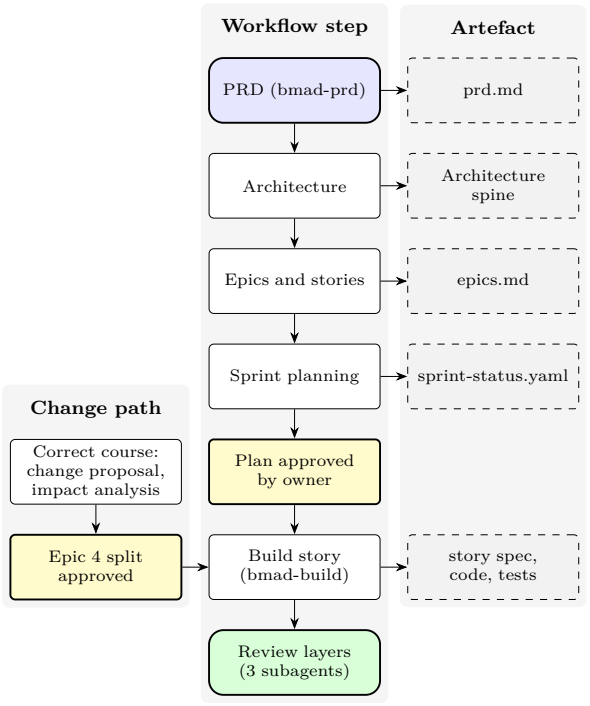


Fig. 2. BMAD as used in the BMAD track (version 6.12.4): workflow steps, the artefact each step leaves, and the change path.

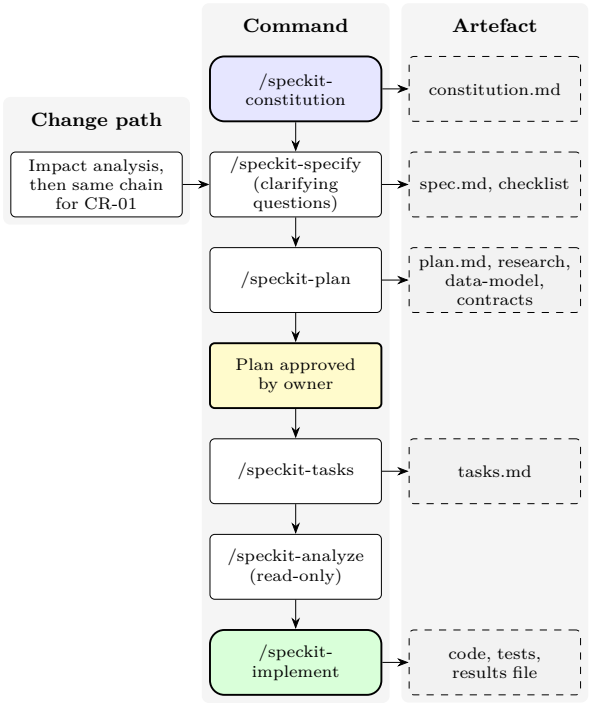


Fig. 3. GitHub Spec Kit as used in the Spec Kit track (specify-cli 1.2.3): commands, the artefact each leaves, and the change path.

The PRD holds ten functional and five non-functional requirements. The architecture spine records ten architecture decisions. Work was split into three epics and nine stories, each built with bmad-build and checked by three review layers, except two stories where the operator chose a minimal path. For the change request, bmad-correct-course produced a sprint change proposal and an impact analysis before any code; after the owner approved a new Epic 4 with four stories, the PRD, architecture and epics were amended and the stories were built.

B. GitHub Spec Kit

GitHub Spec Kit is an open-source toolkit that gives AI coding agents structured processes, reusable templates and documented outcomes [4]. Its core SDD process runs in the agent's chat: /speckit-constitution records project principles once, /speckit-specify writes the specification, /speckit-plan turns it into a technical approach, /speckit-tasks derives tasks, and /speckit-implement produces code; a convergence command can repeat until the result matches the plan. The installed version also provides skills for clarification, checklists and a read-only consistency analysis.

Fig. 3 shows the path actually followed. The constitution was drafted from the shared documents and amended once for the change request (version 1.0.0 to 1.1.0) with the owner's approval. /speckit-specify raised three clarification questions, which the owner answered. The plan produced a research file, a data model, contracts and a quickstart; the owner approved the plan before /speckit-tasks generated 34 baseline tasks. /speckit-analyze reported 11 findings (none critical, two high, two medium, seven low) before any code, and fixes were applied after the owner's approval. For CR-01 the track wrote an impact analysis first and then ran the same chain in a new feature folder, with 49 tasks and a second analysis of 10 findings.

C. Capability Comparison

Table III compares what the two workflows offered, on the criteria of the project brief. Entries describe what the artefacts and logs show; they are not ratings.

V. Results

All counts in this section are as recorded in the tracks' evidence files and in the team's consolidated results; we did not re-run any test or launch either application. Fig. 4 summarises the acceptance outcomes.

A. Specification Structure and Traceability

Both workflows produced structured, linked artefacts before coding (RQ1, RQ2). BMAD's chain runs from the PRD and the architecture decisions to epics, stories, tests and a requirement-to-test table. Spec Kit's runs from the constitution and the specification to the plan, tasks, tests and a results file with a traceability table. Both contain candidate links from requirement to code and tests. A verified count of native links, which would give a traceability measure (M03), has not been established for either track, and links rebuilt afterwards by an evaluator

TABLE III
Capabilities of the two workflows as observed in the tracks.

Criterion	BMAD	Spec Kit
Specification structure	PRD, architecture decisions, epics, story specs	Constitution, spec, plan, research, data model, contracts, tasks
Requirement-to-task link	Epics and stories; requirement-to-test table in the evidence	Requirement IDs tagged in the spec; "Requirement Coverage" table added to the plan (not native)
Ambiguity handling	Agent questions at start; review layers; gaps raised in the log	Clarification questions in /speckit-specify; /speckit-analyze before code
Acceptance criteria	Shared criteria; a missing NFR criterion fixed at the readiness gate	All 19 criteria mapped to tasks
Test generation	Tests per story; review layers added test patches	Tests as tasks; red runs recorded before code
NFRs	NFR-1 to NFR-5 in the PRD	Baseline NFR IDs tagged in the spec
Change handling	bmad-correct-course, change proposal, new epic, amended PRD and architecture	Impact analysis, new feature folder, amended constitution, new spec, plan and tasks
Human control	Owner approvals; standing approval per story	Owner approvals recorded as AP-01 to AP-06; gates before tasks and code

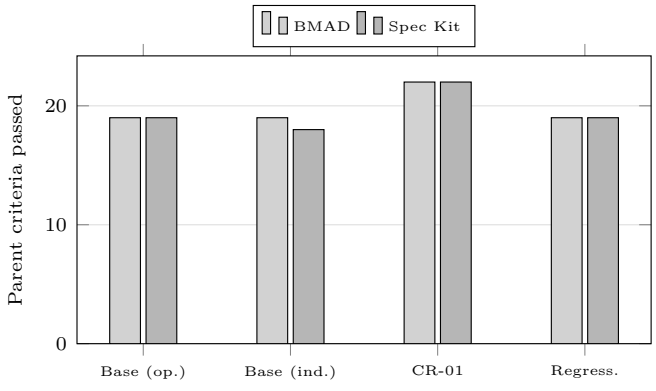


Fig. 4. Parent acceptance criteria passed. Base (op.): baseline, operator report, of 19. Base (ind.): baseline, independent evaluator, of 19 (Spec Kit also had one Blocked). CR-01: of 22. Regress.: baseline re-check after CR-01, of 19.

(M13) have to be kept apart from it. We therefore do not report a traceability percentage.

The shared requirements were not complete enough to be implemented as written. Both operators had to resolve ambiguities through the common log: what happens when no operation is selected, how ties are rounded for negative numbers, whether .5 and 5. are valid, and when a shown result is cleared. Both tracks found gaps independently. The change request also had gaps that surfaced when the BMAD track read it, among them whether the Second number field stays visible and how an empty input is reported. Gaps were therefore found in both phases, by operating the workflows, not by a review of the requirements.

B. Baseline Implementation

Both operators reported all 19 baseline criteria as passed. The independent evaluation differs (RQ4). The BMAD evaluator, who recorded results on a reported clean checkout, reported 19 passes, but the offline-launch criterion has no evidence that networking was disabled and criterion-level screenshots and logs were not attached. The Spec Kit evaluator recorded 18 passes, no failures and one Blocked criterion: the offline launch was not executed because the network prerequisite could not be established. That Blocked criterion is not a product failure, and the difference between 19 and 18 reflects recorded coverage and evidence provenance, not demonstrated correctness. The Spec Kit evaluation retained real-window automation output, screenshots and observation logs.

The reported examples cover the four operations, decimal precision and rounding, negative inputs, division by zero, invalid and missing operands, Clear, recovery after an error, sequential calculations and the Enter key. They do not show exhaustive correctness or that every NFR is satisfied.

C. Change Request CR-01

Both tracks reported all 22 change criteria as passed and all 19 baseline criteria as passed again after the change, with no regression in the checked set (RQ3, RQ4). The two tracks' evidence is not equally strong. The Spec Kit evaluation retained a frozen-snapshot build output, GUI observations and screenshots, and its offline check was first Blocked and later marked Pass through a human-reported follow-up; both records stay visible. The BMAD evaluator's change results are recorded in the evaluator sheets, but the exact frozen checkout and criterion-level raw observations were not established to the same standard. The final 22 of 22 and 19 of 19 are combined reported statuses, not 22 independently reproduced evidence bundles.

The 22 change criteria test new behaviour and the 19 re-checks test preserved behaviour, so the numbers do not form a series that grows from 19 to 22. As context only, the cited native test suites grew from 172 to 316 tests for

BMAD (with 80 window tests skipped in a headless run and none failing on a display) and from 81 to 116 for Spec Kit with no failures; the two suites differ in design and cannot be compared as quality.

Table IV shows change handling by dimension. Both impact analyses were useful but incomplete. The Spec Kit forecast named five source files that were expected to change and five that were expected to stay unchanged, and that classification held; later, work beyond the forecast appeared: a constitution update, a changed OperationTest.java and an extra helper for error-area size. The BMAD track met GUI operation-button tests and an initial-state helper that its analysis had not listed. A product defect, a wrong test input or expectation and a documentation edit are different units, so counting them against each other would mislead.

TABLE IV
Change handling in CR-01 by dimension.

Dimension	BMAD	Spec Kit
Change planning	Story- and architecture-driven impact analysis; review layers	Impact forecast, then spec, plan and tasks
Unplanned effect	GUI operation-button tests; initial-state helper	Constitution update; OperationTest.java; error-area helper
Product defect found	None established in the case descriptions	Negative square-root message clipped in the GUI; found by visual review, then fixed
Test or documentation correction	Test input 1e41 against 1e40; test-only review patches; one Javadoc correction	One wrong test expectation corrected
Impact recall (M07)	Not calculable	Not calculable
Rework (M12)	Candidate episodes only	Candidate episodes only

D. Process Observations

Several observations come from the process records of single runs and carry no statistical weight. In the BMAD track, the review layers returned many items per story, and about half of them were verified as false or rejected as low priority, so every item had to be triaged. The story split by architecture layer made Epic 2 mostly a verification exercise instead of new work. In the Spec Kit track, /speckit-analyze found coverage and consistency gaps before any code, and they were fixed after approval; the one product defect of the change phase was found by looking at the real window, not by an automated test.

Both tracks recorded deviations from their tool's native process, listed in the evidence files. The BMAD track used a fast path, skipped the UX step, did not run the PRD and architecture reviewer subagents, had the agent judge the readiness gate, and replaced per-story approval by a standing approval. The Spec Kit track did not run a template-resolve step for two skills, added a "Requirement Coverage" table to the plan that the tool does not produce, and left one gap, the square root of negative zero, unspecified by the owner's decision. Both tracks also had documentation drift: a BMAD status file still described the change as uncommitted after it had been committed, and a Spec Kit plan referred to a log entry before it existed.

Table V states which measures could be calculated. Most of them could not.

TABLE V
Disposition of the catalogue measures in this study.

Measure	Construct	Status in this study
M01	Parent acceptance outcomes	Calculable as pass ratios per phase; subcase-level evidence incomplete
M02	Requirement-level status	Partial; no verified numeric status
M03 / M13	Native / evaluator-reconstructed traceability	Not scored; semantic link verification pending
M04 / M05	Product defects / late-defect share	Descriptive examples only; no complete census
M06	Baseline regressions after CR-01	Zero reported; observation provenance unequal
M07	Change-impact omission and recall	Not calculable; no adjudicated reference set
M08 / M12	Human interventions / rework episodes	Illustrative only; no comparable census
M09 / M10	Active effort / elapsed duration	Not calculable; no measured sessions
M11 / M14	Rubric ratings J1-J7 / reviewer agreement	Not rated; no composite score
M15	Evidence completeness	Not calculable as a percentage
M16	Native test inventory	Context only; no cross-track quality inference

VI. Discussion

A. Answers to the Research Questions

RQ1 (specification structure). The tools structure specifications differently. BMAD builds a layered set of planning documents (PRD, architecture decisions, epics, stories) around what the system should do and why it is designed as it is. Spec Kit builds a chain of principles, specification, plan and tasks around what is specified and which tasks follow. In both, the human approved plans and answered questions, and in both the operators had to settle ambiguities in the shared requirements through the log. We cannot say which structure is better; no rubric ratings of specification clarity exist.

RQ2 (traceability). Both produced artefacts from which a requirement can plausibly be followed into tasks, code and tests, and each track kept a requirement-to-test table.

We could not verify native link coverage, so the answer is limited to saying that both made links available and that Spec Kit's coverage table was an addition to its plan, not a tool output.

RQ3 (change handling and alignment). Both tracks produced a pre-coding impact analysis, updated their planning artefacts, implemented CR-01 and reported no regression. BMAD's path went through a change proposal and an amended PRD, architecture and epics; Spec Kit's through a new feature folder and an amended constitution. Neither impact analysis was complete, and an adjudicated reference set for impact recall does not exist, so we cannot rank them. Whether specification and code stayed aligned was checked in both tracks only at the end of the change, in line with the late spec-code checking that practitioners report [1]. That observation is about our two runs; it is not a finding about the tools.

RQ4 (match to requirements). Both implementations reported full success on the shared criteria, and no independent run observed a product failure. The independent baseline difference of one Blocked criterion is a difference in evidence. Four NFRs have no linked acceptance criterion, so the evaluation says little about them. Passing 19 of 19 or 22 of 22 parent criteria does not establish exhaustive correctness.

Taken together, the answers are dimension-specific. Table VI lists what the evidence supports and where it stops. We make no claim about which workflow performed better overall, handled the change better or produced better documentation, because effort, impact recall, rework, native traceability and rubric ratings were not measured, and observation layers differ between the tracks.

TABLE VI
Strengths and constraints per workflow, with evidence limits.

Workflow	Supported strengths	Constraints and evidence limits
BMAD	Coherent PRD, architecture, epic and story organisation; review-oriented change handling; explicit impact analysis	Unplanned GUI test and helper work; test-input and documentation corrections; independent change evidence less complete
Spec Kit	Constitution, spec, plan and task chain; impact forecast with changed and unchanged files; retained independent build and GUI evidence; defect found and fixed	Unplanned constitution, test and helper changes; a GUI defect before correction; offline check first Blocked

B. Implications

Four practical points follow from the study, for this case and not for all projects. A change should be judged against new behaviour and preserved behaviour together, because the two sets of criteria answer different questions. Planning and impact documents should be followed by a check in the real window, because the clipped error message passed automated tests. Keeping the original Blocked attempt next to a later manual completion avoids hindsight bias. Finally, an independent evaluation needs a frozen commit, the commands, timestamps and the relevant environment state, and a measure such as impact recall needs a prediction fixed before coding and a reference set agreed afterwards. Effort, rework and rubric data would have to be collected while the work is done; they cannot be reconstructed from commit times.

C. Threats to Validity

Construct validity. A parent Pass does not prove that every subcase was independently executed, and native test counts are not acceptance coverage. We keep these layers separate in every table.

Internal validity. The tool, the agent and the operator are confounded: one operator per tool, two operating systems, and a model change at the start of the Spec Kit track. The study owner operated the BMAD track and wrote the change request, which may bias the BMAD track toward familiarity with the shared documents. The comparison is descriptive and not a randomised benchmark.

Reliability. Each workflow was run once, so we have no estimate of the run-to-run variability of the agent. The BMAD change-phase evaluation lacks an archived frozen checkout and raw observations of the same depth as the Spec Kit package. The Spec Kit baseline evaluation used a snapshot designated by the owner, not declared by the operator. Neither track saved every prompt verbatim.

Conclusion validity. Missing measures are not treated as zero, and no composite score is computed. One small application, one change request and two workflows do not support claims about SDD tools in general.

Shared input. The ambiguities in the baseline and in CR-01, the four NFRs without acceptance criteria and the three criteria without requirement IDs affect both tracks equally, but they limit what the evaluation can show.

VII. Conclusion

Within the limits of one case, both workflows carried the same requirements through specification, planning, tasks, code and tests, handled the same change request with a pre-coding impact analysis, and reported all 22 change criteria and all 19 baseline re-checks as passed. The independent baseline results, 19 passes for BMAD and 18 passes with one Blocked for Spec Kit, differ in evidence provenance and not in demonstrated correctness. Spec Kit's independent change evaluation was more inspectable, and BMAD's planning documents gave a continuous account of architecture and story decisions. Both required human decisions to settle ambiguities in the

shared requirements, and both met effects of the change that their impact analyses did not list. We found no basis for an overall ranking. Future work should fix an impact reference set before coding, measure effort and rework prospectively, collect independent ratings, repeat the runs, and verify native traceability link by link.

VIII. AI-Use Declaration

AI tools were used in four ways. (1) Both calculators were built with Claude Code and the model claude-sonnet-5-5, which acted as the operator's coding agent in each track; the Spec Kit track started on claude-opus-5-5 and switched to claude-sonnet-5-5. The agent produced the planning artefacts, code, tests and evidence drafts through the BMAD 6.12.4 skills (including review subagents) and the Spec Kit 1.2.3 skills; the operators approved plans and recorded decisions. (2) The Spec Kit baseline evaluation was run with the assistance of an AI coding assistant, as recorded in the experiment log; the files we read do not state whether an assistant was also used in the Spec Kit change evaluation. (3) The literature registers were produced with AI-assisted source discovery and drafting, and an evaluation skill (evaluate-sdd-workflows) was used to prepare evaluations. (4) This report was drafted with Claude Code and a document-production workflow, from the team's repository files and from bibliographic records and abstract pages fetched on 2026-11-06; every reference was checked against such a record, and no full text of the cited works was read. [[EVIDENCE NEEDED: confirmation by the authors that the text of this report was reviewed and approved by a human author, and the exact wording required by the course for the AI-use declaration]]

References

[1] S. Agarwal, A. Singhal, A. Macvean, B. Vasilescu, and T. Breaux, "Spec-driven development as prescribed and as practiced," arXiv preprint arXiv:2608.25202 [cs.SE], 2026, version 4, 15 October 2026.

[2] D. B. Piskala, "Spec-driven development: From code to contract in the age of AI coding assistants," arXiv preprint arXiv:2602.00180 [cs.SE], 2026, submitted 30 January 2026.

[3] B. Claudino, R. Xing, Y. Zuo, T. Menezes, A. Sarma, K. Damevski, and Z. Feng, "The EPIC framework for spec-driven development," arXiv preprint arXiv:2610.07534, 2026, submitted 6 October 2026; peer-review status not established.

[4] GitHub, "Spec kit," https://github.com/github/spec-kit, 2026, tool documentation, accessed 2026-11-06.

[5] bmad-code-org, "BMAD-METHOD," https://github.com/bmad-code-org/BMAD-METHOD, 2026, tool documentation, accessed 2026-11-06.

[6] B. Hill, "Specification artifacts in open-source pull request workflows do not reduce defects: An empirical test of spec-driven development claims," in Proc. 20th Int. Symp. Empirical Software Engineering and Measurement (ESEM 2026), ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 394. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, pp. 84:1–84:20.

[7] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," Empirical Software Engineering, vol. 14, no. 2, pp. 131–164, 2009.

[8] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, and J. Rosenberg, "Preliminary guidelines for empirical research in software engineering," IEEE Transactions on Software Engineering, vol. 28, no. 8, pp. 721–734, 2002.

[9] V. R. Basili, G. Caldiera, and H. D. Rombach, "Goal question metric approach," in Encyclopedia of Software Engineering. John Wiley & Sons, 1994, pp. 528–532.